

```

root@ubuntu-vm:/home/ubuntu/Projects/make-sense-of-containers# go run main.go run bash
Running [bash] as PID 1
root@container:/#

root@ubuntu-vm:/sys/fs/cgroup/pids# ls
cgroup.clone_children  cgroup.sane_behavior  pratikms  system.slice  user.slice
cgroup.procs          notify_on_release     release_agent  tasks
root@ubuntu-vm:/sys/fs/cgroup/pids# cd pratikms/
root@ubuntu-vm:/sys/fs/cgroup/pids/pratikms# ls
cgroup.clone_children  cgroup.procs  notify_on_release  pids.current  pids.events  pids.max  tasks
root@ubuntu-vm:/sys/fs/cgroup/pids/pratikms# cat pids.max
20
root@ubuntu-vm:/sys/fs/cgroup/pids/pratikms# cat pids.current
6
root@ubuntu-vm:/sys/fs/cgroup/pids/pratikms#

```

Figure 5: Confirming creation of Cgroups for our process

```

Running [bash] as PID 1
root@container:/# :() { : | : & }; :
[1] 14
root@container:/# bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
root@ubuntu-vm:/sys/fs/cgroup/pids# ls
cgroup.clone_children  cgroup.sane_behavior  pratikms  system.slice  user.slice
cgroup.procs          notify_on_release     release_agent  tasks
root@ubuntu-vm:/sys/fs/cgroup/pids# cd pratikms/
root@ubuntu-vm:/sys/fs/cgroup/pids/pratikms# ls
cgroup.clone_children  cgroup.procs  notify_on_release  pids.current  pids.events  pids.max  tasks
root@ubuntu-vm:/sys/fs/cgroup/pids/pratikms# cat pids.max
20
root@ubuntu-vm:/sys/fs/cgroup/pids/pratikms# cat pids.current
6
root@ubuntu-vm:/sys/fs/cgroup/pids/pratikms# cat pids.current
20
root@ubuntu-vm:/sys/fs/cgroup/pids/pratikms#

```

Figure 6: Executing a fork bomb

```

3327 pts/1    Sll    0:00 | \_ go run main.go run bash
3360 pts/1    Sll    0:00 | \_ /tmp/go-build432951705/b001/exe/main run bash
3365 pts/1    Sll    0:00 | \_ /proc/self/exe child bash
3370 pts/1    S+    0:00 | \_ bash
3421 pts/1    Z     0:00 | \_ [bash] <defunct>
3422 pts/1    Z     0:00 | \_ [bash] <defunct>
3423 pts/1    Z     0:00 | \_ [bash] <defunct>
3424 pts/1    Z     0:00 | \_ [bash] <defunct>
3425 pts/1    Z     0:00 | \_ [bash] <defunct>
3426 pts/1    Z     0:00 | \_ [bash] <defunct>
3427 pts/1    Z     0:00 | \_ [bash] <defunct>
3428 pts/1    Z     0:00 | \_ [bash] <defunct>
3429 pts/1    Z     0:00 | \_ [bash] <defunct>
3430 pts/1    Z     0:00 | \_ [bash] <defunct>
3431 pts/1    Z     0:00 | \_ [bash] <defunct>
3432 pts/1    Z     0:00 | \_ [bash] <defunct>
3433 pts/1    Z     0:00 | \_ [bash] <defunct>
3434 pts/1    Z     0:00 | \_ [bash] <defunct>

```

Figure 7: Observing output of `ps fax`

on our container.

```
:() { : | : & }; :
```

No, this is not a typo. Neither did you read it wrong. It is essentially a fork bomb. A fork bomb is a denial-of-service attack wherein a process continuously replicates itself to deplete

available system resources, slowing down or crashing the system due to resource starvation.

To make more sense of it, you can literally replace the `:` in it with anything. For example, `:() { : | : & }; :` can also be written as `forkBomb() { forkBomb | forkBomb & }; forkBomb`. It means that we're declaring a function

`forkBomb()`, the body of which recursively calls itself with `forkBomb | forkBomb` and runs it in the background using `&`. Finally, we call it using `forkBomb`. While this works, a fork bomb is conventionally written as `:() { : | : & }; :`, and that is what we will proceed with.

As we can see in Figure 6, the current number of processes running within our container were six. After we triggered the fork bomb, the current number of running processes increased to 20 and remained stable there. We can confirm the forks by observing the output of `ps fax` in Figure 7.

Putting it all together

So here it is — a super simple container, in less than 100 lines of code. Obviously, this is simple. If you use it in production, you are crazy and, more importantly, on your own. But I think seeing something simple will give you a really useful picture of what's going on. In case you missed anything from the first part of this two-part series, this code snippet can serve as a reference.

```
package main
```

```
import (
    "fmt"
    "io/ioutil"
    "os"
    "os/exec"
    "path/filepath"
    "strconv"
    "syscall"
)
```

```
func must(err error) {
    if err != nil {
        panic(err)
    }
}
```

```
func cg() {
    cgroups := "/sys/fs/cgroup/"
    pids := filepath.Join(cgroups,
        "pids")
    err := os.Mkdir(filepath.Join(pids,
```